Running a 260K-Parameter Llama 2 Inside the Octra VM

qiubit

October 15, 2026

Abstract

We ported `stories260K`, a 260K-parameter Llama 2 model (MIT), to an AML program that runs transformer inference inside the Octra VM. Weights are stored as int8 with per-row scales, and all arithmetic uses the VM's Q16 fixed-point builtins. We deployed two builds on Octra devnet. Tokens produced on-chain matched the float32 reference in every test. One call processes at most 9 tokens without a prompt, or 10 positions when some of them are prompt tokens. On-chain capacity tracks the number of vector operations (between roughly 31 and 35 thousand `vecdot_q16` calls per call), not the effort reported by the local VM. Transactions did not raise the limit: a `program_exec` call to `generate` reverted even at 5 tokens. Setup costs about 0.3 OCT per deployment, and inference through view calls is free. Computation is plaintext, so prompts and outputs are public.

1 Goal

Octra offers two computation paths for programs. The HFHE path works on ciphertexts, but AML only exposes addition, subtraction, and multiplication by a plaintext scalar. The official `private_ml` example is therefore a linear model $w \cdot x + b$. In our local benchmark with `libpvac`, a single 512-element dot product took 71 seconds and produced a 22 MB ciphertext. The second path is the plaintext VM with Q16 vector builtins (`vecdot_q16`, `rmsnorm_q16`, `rope_apply_q16`, `attention_kv_q16`, `silu_q16`). We used the plaintext path to answer one question: how large a language model can run end to end inside a single call?

2 Model

Architecture	Llama 2: RMSNorm, RoPE, SwiGLU, grouped-query attention
Size	dim 64, FFN 172, 5 layers, 8 heads, 4 KV heads, vocabulary 512
Parameters	about 260 thousand
Training data	TinyStories (simple English children's stories)

Weights and tokenizer come from https://huggingface.co/karpathy/tinyllamas (MIT). We chose this model because it shares SmolLM2's architecture, and its weights, tokenizer, and reference code (https://github.com/karpathy/llama2.c) are all MIT-licensed. The `stories15M` and larger checkpoints use Meta's Llama 2 tokenizer and fall under the Llama 2 Community License. The TinyStories dataset is CDLA-Sharing-1.0, whose share-alike terms apply to redistributing the data; a trained model counts as "Results". The model can only continue simple children's stories.

3 Port

Quantization. Each row W_r is stored as int8 values q_{rj} with a Q16 scale $s_r = \text{round}(\max_j |W_{rj}|/127 \cdot 2^{16})$. The VM loads q_{rj} as $q_{rj} \cdot 2^{16}$, so `vecdot_q16` computes $\sum_j q_{rj} x_j$ with no truncation. Each row result is then multiplied by s_r with `elemwise_mul_q16`.

RoPE. `llama2.c` rotates pairs $(2j, 2j{+}1)$, while `rope_apply_q16` rotates $(j, j{+}d/2)$ at the same frequency. We permute the rows of W_q and W_k inside each head to the order $[0, 2, 4, 6, 1, 3, 5, 7]$. Applying the same permutation to q and k leaves $q \cdot k$ unchanged.

Node compiler rules. The devnet node compiler (`2.0 AML-preview`) rejects many builtins that the public `amlc` accepts. Our probes found:

- Rejected: `matmul_q16`, `matmul`, `matmul_fp`, `vecdot`, `parse_ints`, `shift_round`, `load_int8_b64`.
- Accepted: `vecdot_q16`, `rmsnorm_q16`, `rope_apply_q16`, `attention_kv_q16`, `silu_q16`, `elemwise_mul_q16`, `residual_add_q16`, `append_vec_q16`, `argmax_q16`, `load_int8_q16`, `mset`.
- Values returned by `mget` have type *unknown* and cannot enter arithmetic.

As a result, every matrix-vector product is written as one `vecdot_q16` per row. A single memory cell is read with `vecdot_q16(addr, ONE_CELL, 1)`, where `ONE_CELL` holds 2^{16}. Scales, gammas, and the prompt are sent as base64 int8 planes because `parse_ints` is unavailable.

Flow. The owner deploys the program and uploads 83 tensors (362,716 base64 characters) through `set_tensor`. A client tokenizes text locally and calls `generate(prompt, n, steps)` as a view call. The program loads the weights from storage, runs a greedy forward pass per token with the KV cache in memory, and returns token ids. The qiubit web page (`web/`) does this from the browser. The devnet RPC allows cross-origin requests, and a view call takes any well-formed caller address, so users need no wallet.

4 Results

Equivalence. We compared four implementations: the float32 forward pass, a Python emulator that reproduces `fixed_q16.ml` bit for bit, the local `amlc` VM, and the devnet node. Without a prompt, the three local implementations produce the same text as the model card ("Once upon a time, there was a little girl named Lily…"). For 60 tokens, the SHA-256 of the token-id CSV is identical across them: `fb5c2a41751ad940d7646b006a973aace7d860f5343448def941a5dbe1313b7a`.

On-chain evidence. The data below comes from https://devnet.octrascan.io/rpc. Build v1 is the first version. Build v2 adds two optimizations: the `lin` loop is unrolled four rows per iteration, and the 512-row classifier is skipped at positions whose next token is already known from the prompt.

v1 (`aml/tinystories_deployed.aml`)	
Program	`octHx7PLvqUka2CMVHVYXReAwfvwzv15uuoYh9qqAKVB1rj`
`code_hash`	`7653be39bbfab88d48e7ebb7ac0687e400aac2c11294d80cb44aa0b3d33215c6`
Deploy tx	`ca0d5205cf38a537ec32580a9946c3cb9ce10fb50862e3f096f626f4ffc25f12`, epoch 1,695,023
v2 (`aml/tinystories_v2.aml`)	
Program	`oct4LetvjEP7fzTyvaUBSu7aDKBtM1peufyCBmSDvos5HTR`
`code_hash`	`84ad4f9bfc4d95a937d31a13a850e2bf74fb0c70e191ce58572b6fefa74d06c9`
Deploy tx	`dbd71ee415b53e559c71cce6052b700d25df6d3028347c6f7fe451689449b72e`, epoch 1,695,549
Weight upload	83 `set_tensor` txs per build, every receipt `success: true`

For both builds, the SHA-256 of the bytecode compiled from the included source equals the `code_hash` reported by the node, so that source is the program running on-chain.

Build	Prompt (tokens)	Positions	On-chain result
v1	BOS (1)	9	match, `403,...,376`
v1	“Once upon a time, Tom” (8)	9	match, `...,269,410`
v1	BOS (1)	10	revert
v2	BOS (1)	9	match
v2	BOS (1)	10–14	revert
v2	“Once upon a time, there was a little” (10)	10	match
v2	“... a little girl named” (14)	12	revert
v2, tx, 3M OU	BOS (1)	12	`rejected`: revert
v2, tx, 3M OU	BOS (1)	5	`rejected`: revert

Every successful run matched the Python reference. The second row contains 7 prompt tokens processed one at a time, then 2 new tokens: “ and” and a space.

Capacity and cost. Effort as counted by `amlc` does not predict the on-chain limit. Build v1 passed 9 tokens at 2,014,286 local effort. Build v2 reverted at 10 tokens even though its local effort was only 1,777,450 and its VM step count 346,445, both lower than v1 at 9 tokens (413,505 steps). The measure that fits every result is the number of `vecdot_q16` calls per call. Runs with 31,608 calls (v1, 9 tokens) and 30,512 calls (v2, 10 positions with a prompt) passed, while 35,120 (v2, 10 tokens) and 36,000 (v2, 12 prompt positions) reverted. Every vector operation in `contract_vm.ml` reserves memory through `reserve_vectors`, and that allocation charge (`byte_work`) is disabled in the local VM. We believe this charge is what binds on the node.

So the unrolling in v2 cut scalar instructions by about 25% without adding on-chain capacity. What added one position was skipping the classifier, which saves 512 `vecdot_q16` calls per prompt position. Calling `generate` through a `program_exec` transaction with 3 million OU reverted at both 12 and 5 tokens. Either the node does not run `view` functions in transactions, or transactions have a smaller budget; the current program cannot tell these apart. Reverted transactions did not reduce the balance. Each deployment cost 219,000–220,000 OU plus 83,000 OU for the weight upload, so the two deployments cost 605,000 OU (0.605 OCT) in total. View calls need no fee, and a 9–10 position response arrives in 4–13 seconds.

5 Limitations

- One call processes at most 9 tokens without a prompt, or 10 positions when some are prompt tokens.
- Results are not recorded on-chain. A view call is computed by the single node that answers it, and the transaction path reverted in our tests.
- View calls keep no state. To continue a story, the client must resend the earlier text as the prompt, and that prompt consumes the same budget.
- Weights are reloaded on every call because VM memory does not persist between calls.
- Local VM effort is not enough to estimate the on-chain limit. Capacity has to be measured on the node.
- Prompts, outputs, and weights are public. The HFHE path supports only linear models.
- SmolLM2-135M needs about 500 times more operations per token than this model, so a model that size does not fit in one call.
- The devnet RPC fronts nodes that are out of sync. Clients must retry requests that fail with `bytecode not found` or `Program view busy`, and `webcli` can sign transactions with a stale nonce. Devnet also stopped producing epochs for several hours during our work.

There are three next steps. First, the node could admit `matmul_q16`, replacing 3,512 `vecdot_q16` calls per token with about 36 matmul calls; we have not measured how the node charges for them. Second, the Octra team could publish the per-call allocation limit so capacity can be computed instead of found by trial and error. Third, a program with a non-view entry point that stores its output and the KV cache in storage would show whether transactions can run inference and record results; each step would still be limited to about 9 positions and pay for 320 storage writes per token.

Sources and licenses

- `stories260K` weights and tokenizer, MIT: https://huggingface.co/karpathy/tinyllamas
- `llama2.c`, MIT: https://github.com/karpathy/llama2.c
- TinyStories, CDLA-Sharing-1.0: https://huggingface.co/datasets/roneneldan/TinyStories. Paper: R. Eldan and Y. Li, https://arxiv.org/abs/2305.07759
- `amlc`, BSD-3-Clause: https://github.com/octra-labs/amlc
- `webcli`, GPL-2.0-or-later, used only to sign and send transactions: https://github.com/octra-labs/webcli
- AML documentation: https://docs.octra.org/developer-docs/introduction-to-applied
- Code, the web page, and raw evidence (`build/evidence_v1.json`, `build/evidence.json` for v2, `build/evidence_tx.json`, `build/upload*.json`) are in the project repository.